



I E S A L B A R R E G A S

TÉCNICO SUPERIOR EN DESARROLLO DE
APLICACIONES WEB

Departamento de Informática

MOTORSPORTSLIVE

Manual Técnico

Índice

1. **Introducción**
 - 1.1. **Objetivo del proyecto**
 - 1.2. **Motivación temática**
 - 1.3. **Tecnologías utilizadas**
2. **Arquitectura de la Aplicación**
 - 2.1. **Entorno de desarrollo y herramientas**
 - 2.2. **Backend**
 - 2.3. **Frontend**
3. **Documentación Técnica**
 - 3.1. **Análisis de requisitos**
 - 3.1.1. **Requisitos funcionales**
 - 3.1.2. **Requisitos no funcionales**
 - 3.2. **Desarrollo del sistema**
 - 3.2.1. **Modelo Entidad-Relación**
 - 3.2.2. **Modelo relacional**
 - 3.2.3. **Diagrama de casos de uso**
 - 3.3. **Pruebas realizadas**
 - 3.3.1. **Pruebas funcionales**
 - 3.3.2. **Pruebas de interfaz y rendimiento**
4. **Proceso de Despliegue**
 - 4.1. **Herramientas utilizadas**
 - 4.2. **Despliegue en Vercel**
 - 4.3. **Configuración de entorno (.env)**
 - 4.4. **Repositorio GitHub y actualización automática**
5. **Propuestas de Mejora**
 - 5.1. **Requisitos funcionales pendientes**
 - 5.2. **Mejoras técnicas**
 - 5.2.1. **Backend**
 - 5.2.2. **Frontend**
6. **Webgrafía**
 - 6.1. **Cursos online utilizados**
 - 6.2. **Webs y documentación de referencia**
 - 6.3. **Artículos y enlaces relevantes**

1. Introducción

MotorsportsLIVE es una aplicación web interactiva creada por y para seguidores del deporte del automovilismo. Este proyecto nace con el propósito de poder informar a todo el mundo sobre la actualidad de este deporte. Para esta aplicación, me he creado una API, de la cual se obtienen todos los datos históricos de la aplicación.

Esta aplicación, la he realizado inspirándome en otra aplicación llamada BeSoccer, donde se pueden buscar diferentes resultados de fútbol.

Objetivos del Proyecto

A continuación se detallan los objetivos específicos que guían el desarrollo de esta aplicación:

1.1 Crear una experiencia auténtica e intuitiva

Se ha diseñado una interfaz básica y agradable para que cualquier usuario pueda acceder sin tener que complicarse mucho.

1.2 Ofrecer funcionalidades enfocadas al usuario

La aplicación permite:

- Ver datos de carreras, pilotos, equipos, circuitos,...
- Editar, o crear usuarios (sólo para administradores)
- Crear, o editar, datos de F1 (sólo para editores).
- Navegar entre secciones.

Estas funciones se presentan mediante una interfaz intuitiva.

1.3 Garantizar escalabilidad y adaptabilidad a cambios futuros

Gracias al uso de tecnologías modernas como:

- **Node.js** para el backend.
- **Supabase** como backend sin servidor (BaaS).
- **Vercel** para subir la aplicación y la API a un servidor externo.
- **Express.js** para ayudar en el backend.

...la arquitectura resultante es fácilmente ampliable. Esto permite la integración futura de nuevas funciones para todo tipo de usuarios y también para poder introducir datos más detallados relacionados con la Fórmula 1 y con la API en sí.

1.4 Proporcionar una experiencia completa, responsive y accesible

Se ha aplicado un enfoque **mobile-first**, asegurando que todos los componentes y secciones de la app funcionen de forma fluida en casi todos los dispositivos móviles y tabletas. Se implementaron adaptaciones específicas para tamaños $\leq 480\text{px}$.

Público objetivo

MotorsportsLIVE está dirigido principalmente a:

- Fans de los deportes relacionados con el automovilismo.
- Usuarios que buscan obtener información sobre la F1.
- Estudiantes o desarrolladores interesados en ver una integración práctica de tecnologías web modernas aplicadas a un producto temático.

2. Arquitectura de la aplicación

2.1. Introducción

En cuanto a las herramientas empleadas para el desarrollo del proyecto **MotorsportsLIVE**, se han utilizado principalmente las siguientes tecnologías y entornos de desarrollo:

- **Sistema Operativo:**
 - Windows (Windows 11 Pro 24H2)
- **Entorno de Desarrollo Integrado (IDE):**
 - Visual Studio Code (> v1.80.0)
- **Herramientas Frontend:**
 - Bootstrap (5.3.3)
 - CSS personalizado (Media Queries y diseño responsive)
- **Herramientas Backend:**
 - Supabase
 - Autenticación de usuarios
 - Node.js, como backend.
 - Express.js para ayudar a subir la aplicación al servidor.
 - Almacenamiento de archivos multimedia (Storage)
- **Aplicaciones y Herramientas Adicionales:**
 - GitHub (> v3.0.2) para control de versiones
- **Despliegue y Hosting:**
 - Vercel para alojamiento frontend(y backend) y despliegue automático desde GitHub
- **Aplicaciones en Consola de Comandos:**
 - NodeJS (> v18.0.0)
 - npm (> v9.0.0)
 - Git (> v2.40.0)
 - Vercel CLI (> v28.0.0)

En los siguientes apartados detallaremos cuáles han sido las tecnologías específicas utilizadas, así como los patrones de diseño y arquitectura aplicados durante el desarrollo del proyecto.

2.2. Backend

La implementación del backend de MotorsportsLIVE se ha centrado en garantizar una comunicación eficiente y segura entre el cliente y los servicios en la nube proporcionados por Supabase.

Partiendo del principio fundamental de desacoplar claramente las responsabilidades entre cliente y servidor, se han tenido en cuenta los siguientes aspectos arquitectónicos y de diseño:

- **Relación con Supabase:**

El proyecto aprovecha la plataforma Supabase, un Backend-as-a-Service basado en PostgreSQL, que gestiona automáticamente la infraestructura de servidores, lo que simplifica enormemente la escalabilidad, el mantenimiento y las actualizaciones del backend.

- **Modelo Cliente-Servidor basado en REST:**

Para la comunicación entre la aplicación cliente y los servicios de Supabase se han utilizado endpoints REST (Representational State Transfer), los cuales permiten un manejo claro y estructurado de las peticiones HTTP y facilitan el intercambio de información en formato JSON. También he estado haciendo lo mismo pero con mi propia API.

- **Autenticación y autorización mediante JWT (JSON Web Token):**

Supabase proporciona autenticación integrada mediante JWT, que permite gestionar sesiones de usuario seguras y sin estados, facilitando así el intercambio seguro y eficiente de datos entre el cliente y el backend.

- **Estructura de datos y persistencia con PostgreSQL:**

Se emplea PostgreSQL como motor de base de datos, gestionado por Supabase, para persistir información relacionada con usuarios, canciones favoritas y metadatos relacionados.

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto:



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

Las herramientas y características concretas empleadas del servicio Supabase son:

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto:



- **Supabase Database:**

Gestor de bases de datos PostgreSQL integrado para la gestión eficiente y escalable de información como canciones favoritas, usuarios registrados y configuraciones específicas.

- **Supabase Auth:**

Módulo para la gestión segura y sencilla de usuarios mediante registro, inicio de sesión, recuperación de contraseñas y gestión de tokens JWT.

Para el manejo del código JavaScript en la parte frontend que interactúa con el backend, se han desarrollado funciones específicas reutilizables en los módulos:

- [supabase.js](#): Gestiona la conexión inicial y autenticación con Supabase.

Esta elección tecnológica permite una arquitectura simplificada, robusta y fácil de mantener, adecuada para un proyecto web orientado a la experiencia del usuario y a la observación de datos como lo es **MotorsportsLIVE**.

2.3. Frontend

Para el desarrollo de la parte cliente de la aplicación **MotorsportsLIVE**, se han empleado las siguientes tecnologías modernas y ampliamente utilizadas en el ámbito del desarrollo web:

Lenguajes de programación y marcado

- **JavaScript** : lenguaje base empleado para toda la lógica de la aplicación.
- **HTML5**: estructura semántica básica de la aplicación.
- **CSS3**: estilos avanzados y personalizados para la interfaz.

Frameworks y librerías principales

- **JSON Crack**: biblioteca principal utilizada para desarrollar la interfaz de usuario interactiva y eficiente.
- **Live Preview**: facilita el despliegue en local de la aplicación para realizar pruebas.
- **Webpack**: herramienta rápida y moderna para el empaquetado de la aplicación.

Librerías específicas complementarias

- **Supabase JS Client**: Librería cliente para interactuar fácilmente con la plataforma Supabase desde la aplicación.

Diseño y adaptabilidad

- **Responsive Design:** mediante el uso avanzado de CSS Media Queries, la interfaz está completamente adaptada a dispositivos móviles de diferentes tamaños de pantalla (principalmente $\leq 480\text{px}$, $\leq 360\text{px}$, $\leq 320\text{px}$).
- **Estilos personalizados:** Se han desarrollado hojas de estilo CSS específicas, con temática inspirada en una estética sencilla, garantizando coherencia visual y una experiencia minimalista.

Estructura de carpetas principal del frontend

La aplicación sigue una estructura clara. A continuación se presenta la estructura básica de carpetas del proyecto:

MotorsportsLIVE

```
| | | assets
| | | components // Componentes o código no utilizado temporalmente
| | | CSS
| | | HTML // Archivos HTML
| | | JS // Archivos JS (dentro está la carpeta de la API con todos sus
| | | ficheros necesarios)

| | | src
| | | index.html // Página de inicio
| | | package-lock.json
| | | package.json
| | | vercel.json // Archivo de configuración de vercel
| | | webpack.config.js // Archivo de configuración de webpack
```

La elección de estas tecnologías y herramientas ha permitido construir una aplicación interactiva, visualmente atractiva y con un rendimiento óptimo, asegurando así una experiencia satisfactoria para todos los usuarios y seguidores del automovilismo.

3. Documentación técnica

3.1. Análisis

A continuación, se exponen los requisitos funcionales y no funcionales implementados adecuadamente en la aplicación MotorsportsLIVE. Para facilitar la comprensión y organización del contenido, se tomarán como referencia los distintos roles definidos dentro de la aplicación:

3.1.1. Requisitos funcionales

Usuario sin cuenta (invitado)

- Acceder a la pantalla inicial.
- Acceder a , casi, cualquier página pudiendo:
 - Ver cualquier piloto en detalle.
 - Ver resultados de carreras, ver datos de equipos, de un circuito concreto,...
- Navegar por las distintas secciones a través del navbar.

Usuario registrado (autenticado con Supabase)

- Iniciar sesión o registrarse mediante formulario (email y contraseña).
- Acceder a todas las funcionalidades del usuario invitado, o administrador, o editor.
- Acciones de usuarios registrados:
 - Poder crear un editor nuevo (sólo para administradores).
 - Poder eliminar un editor existente, a través de una página de confirmación (sólo para administradores).
 - Crear, o editar, los datos de un piloto en específico (sólo para editores).
 - Crear, o editar, un nuevo evento y/o poder añadir más datos al evento en sí y poder crear resultados (sólo para editores).
 - Poder crear temporadas nuevas (sólo para editores)

3.1.2. Requisitos no funcionales

- **Diseño responsive completo:** la interfaz se adapta automáticamente a diferentes resoluciones de pantalla, con estilos específicos aplicados a móviles ($\leq 480\text{px}$) y escritorio, garantizando una experiencia fluida en todos los dispositivos.
- **Experiencia de usuario intuitiva:** la aplicación utiliza elementos visuales sencillos (utiliza algunos iconos personalizados, pero es un diseño minimalista).
- **Autenticación segura mediante Supabase:** se implementa un sistema de login y registro basado en correo electrónico y contraseña gestionado por Supabase Auth, garantizando una verificación fiable y segura de usuarios.
- **Arquitectura modular y mantenible:** la aplicación cuenta con una estructura clara y fácil de entender con carpetas, lo que permite

escalar y extender nuevas funcionalidades sin comprometer la base del proyecto.

3.2. Desarrollo

En este apartado se presentan los elementos técnicos y estructurales más relevantes desarrollados para la aplicación. Incluye la representación de la base de datos utilizada, la cual ha sido creada y gestionada a través de **Supabase**, una plataforma backend como servicio que proporciona una base de datos **PostgreSQL**, almacenamiento de archivos y autenticación integrada.

3.2.1. Diseño de la Base de Datos

La base de datos de la aplicación ha sido diseñada de forma sencilla pero eficaz, centrada en la relación entre usuarios y canciones favoritas. A continuación se describe su estructura:

Tabla Usuarios

Esta tabla almacena los usuarios que se registran en la aplicación. Fue creada desde el panel SQL de Supabase, y cuenta con las siguientes columnas:

☐	 id uuid	☐	email text	☐	nom... t...	☐	apellidos text	☐	tipoUsuario text	☐	 IdUsuario int8	☐	Contraseña text
--------------------------	--	--------------------------	-------------------	--------------------------	--------------------	--------------------------	-----------------------	--------------------------	-------------------------	--------------------------	---	--------------------------	------------------------

Tabla Editores

Esta tabla almacena todos los usuarios que sean del tipo Editor, fue creada desde el panel SQL de Supabase:

☐	 Id int8	☐	IdEditor int8	☐	CorreoElectronico text	☐	NombreEditor text	☐	created_at timestampz	☐	Contraseña text	☐
--------------------------	--	--------------------------	----------------------	--------------------------	-------------------------------	--------------------------	--------------------------	--------------------------	------------------------------	--------------------------	------------------------	--------------------------

Tabla Favoritos

Esta tabla, en principio, se iba a utilizar para que los usuarios de tipo Invitado (que ya estén registrados), pues pudiesen poner como favoritos algunos eventos, circuitos,...:

☐	 idUsuario int8	☐
--------------------------	---	--------------------------

Tabla Noticias

Esta tabla tampoco se puede usar, pero con esta tabla, los editores podrían guardar las propias noticias que ellos mismos se han creado, y así poder guardarlas en un sitio seguro:

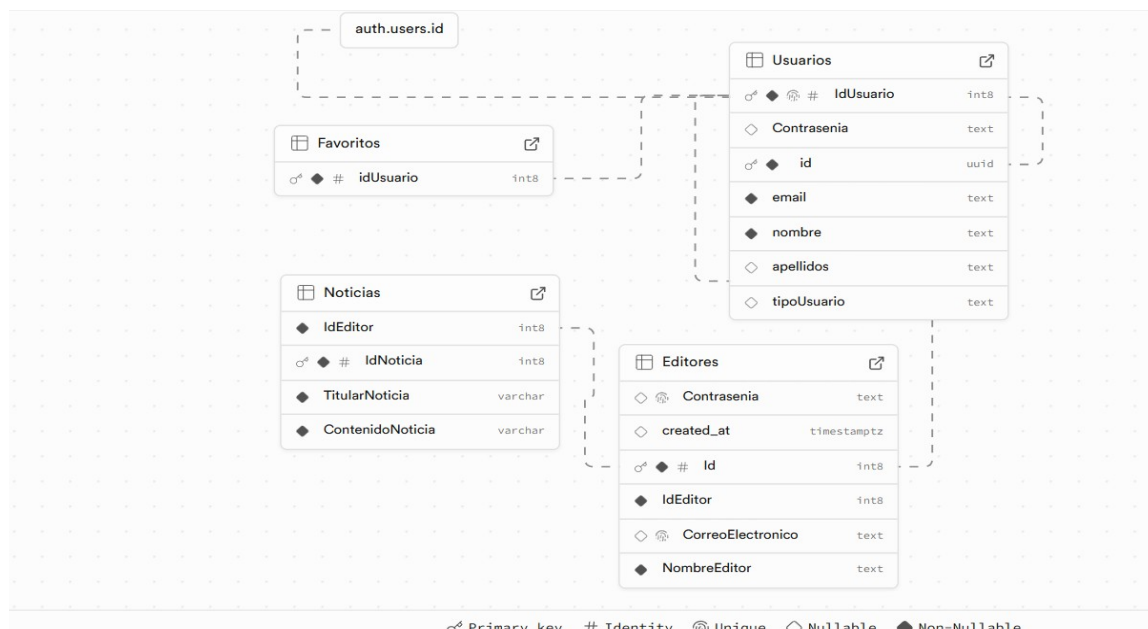
☐	☒ IdNoticia int8	☐ TitularNoticia varchar	☐ ContenidoNoticia varchar	☒ IdEditor int8	☐
--------------------------	--	---	---	---	--------------------------

Relaciones

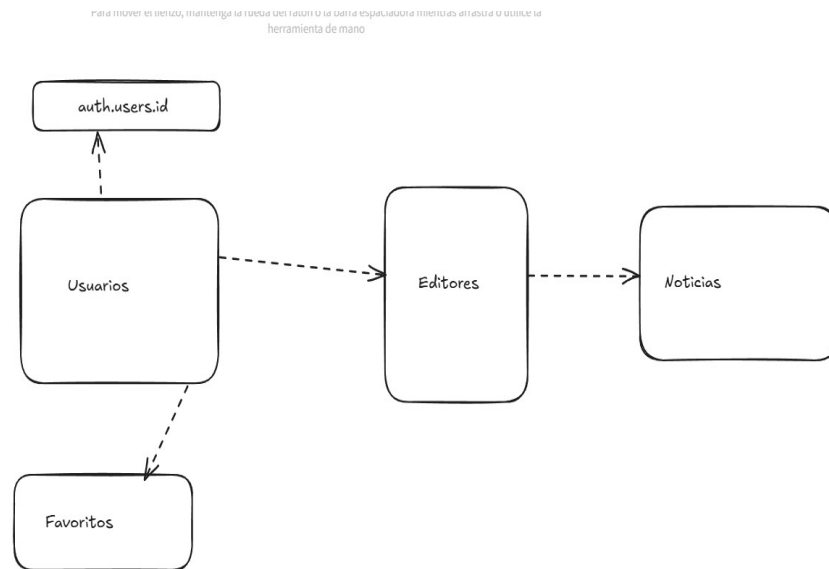
- Relación 1:1 entre **auth.users.id** y **IdUsuario**: cada usuario está relacionado con un id (generado automáticamente al crearse el usuario).
- Relación 1:1 entre **IdEditor** y **IdUsuario**: cada editor está relacionado con un usuario.
- **Restricciones**: **IdUsuario** es clave foránea que referencia al sistema de autenticación interno de Supabase (**auth.users**).

Índice por IdUsuario: mejora el rendimiento de las consultas para recuperar los favoritos de un usuario en concreto.

Modelo Entidad / Relación de la BBDD



3.2.2. Diagrama de casos de uso



3.3. Pruebas realizadas

Para verificar la correcta implementación de los servicios y funcionalidades de la aplicación MotorsportsLIVE, se llevaron a cabo pruebas manuales y exploratorias, principalmente centradas en los endpoints gestionados por **Supabase** y la correcta integración con el frontend.

No he utilizado Supabase como API, porque ya tengo una creada. La he llamada MotorsportsLIVE-API, y se puede encontrar desplegada tanto en Github como en Vercel. A continuación, les paso los enlaces dónde pueden encontrar la API.

- Vercel: [API](#)

- Github: [GitHub](#)

Herramientas de prueba utilizadas

- **Supabase:** Interfaz web para consultar tablas, ver registros insertados, modificar datos manualmente y observar logs en tiempo real.
- **Live Server:** Se utilizó para realizar algunas pruebas relacionadas con el futuro despliegue de la aplicación.
API personal(en local): para comprobar que todos los endpoints funcionan correctamente.

Consola de navegador (DevTools): Útil para verificar errores en tiempo real, validar la respuesta de peticiones fetch y supervisar algún error si es que existiese alguno.

Ejemplo de flujo de pruebas

1. Prueba de inicio de sesión

- Se accede a Supabase Auth mediante el formulario de inicio.
- Se valida la respuesta de Supabase (`usuario !== null`) y que se almacene correctamente en localStorage.
- Se comprueba que el usuario autenticado puede acceder a algún panel, dependiendo de si es un editor, o un administrador.

2. Prueba de funcionalidad como invitado (sin registro, o login)

- Se entra sin tocar nada.
- Se comprueba que el botón de “Iniciar Sesión” se muestra permanentemente hasta que el usuario inicie sesión, o se registre .
- Al darle a alguna opción dedicada exclusivamente a administradores, o editores, sale un mensaje diciendo que no puede hacer eso.

3. Prueba de inserción y eliminación de favoritos (con Node.js)

- Se ejecuta la API manualmente con la ayuda de Node.js.
- Se simulan inserciones vía `POST` a la API `favorites`, utilizando unos datos introducidos válidos.
- Se prueba que los datos aparecen correctamente en la página a visualizar
- Se realiza un `DELETE`, o `INSERT`, o `UPDATE` sobre un registro y se valida que desaparece de la API al recargar.

4. Proceso de Despliegue

4.1. Introducción

Para el despliegue de la aplicación *MotorsportsLIVE*, se ha optado por una arquitectura **frontend centrada** con servicios backend gestionados a través de **Supabase y Node.js**, una solución moderna, eficiente y totalmente integrada con bases de datos PostgreSQL y almacenamiento de archivos.

La aplicación frontend, desarrollada con **HTML y Bootstrap**, ha sido desplegada mediante la plataforma **Vercel**, por su facilidad de uso, integración directa con GitHub y óptimo rendimiento en aplicaciones web.

Esta elección ha simplificado considerablemente el flujo de despliegue, eliminando la necesidad de montar un servidor propio en Heroku o similares. Gran parte de la lógica de backend (autenticación, almacenamiento, y base de datos) se aloja en Supabase, lo que facilita tanto el desarrollo como el mantenimiento, mientras que la otra parte, se de la lógica se almacena en los propios archivos dentro de la carpeta JS.

Además, contar con un repositorio en **GitHub** no solo ha permitido una mejor gestión del código y versiones, sino que también ha sido necesario para enlazar el proyecto con Vercel de forma automática.

4.2. Despliegue del Frontend en Vercel

El despliegue de MotorsportsLIVE se realizó utilizando [Vercel](https://vercel.com), una plataforma moderna para alojar proyectos frontend desarrollados con frameworks como **React** y **Vite**. Esta herramienta destaca por su integración directa con **GitHub**, permitiendo desplegar automáticamente el proyecto con cada nueva actualización en el repositorio.

A continuación, se describen los pasos seguidos para realizar el despliegue:

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto:



1. Acceso e inicio de sesión

Se accede a la página principal de Vercel y se selecciona la opción **“Continuar con GitHub”**. Esto enlaza la cuenta de GitHub del desarrollador con Vercel, permitiendo importar repositorios directamente.

2. Importación del repositorio

Una vez iniciada la sesión, Vercel muestra todos los repositorios públicos y privados asociados a la cuenta de GitHub. Se selecciona el repositorio del proyecto *MotorsportsLIVE*. Posteriormente, se hace lo mismo pero con otro repositorio, en caso de que quieras desplegar otro repositorio en vercel, como hice yo con mi API.

3. Configuración inicial del proyecto

Al importar el repositorio, Vercel detecta automáticamente el framework (en este caso, **ninguno**) y configura las opciones de despliegue. No es necesario modificar la configuración por defecto, salvo definir las **variables de entorno** necesarias para conectar con Supabase.

Estas variables se añaden en el apartado **Environment Variables** dentro de la configuración del proyecto en Vercel.

4. Despliegue inicial

Se hace clic en **“Deploy”** para iniciar la primera implementación. Vercel construye la aplicación automáticamente y despliega el resultado en una URL pública del tipo: [MotorsportsLIVE](#) o [MotorsportsLIVE-API](#)

5. Actualizaciones automáticas

Gracias a la integración con GitHub, cualquier cambio que se realice en la rama principal del repositorio desencadena automáticamente una nueva compilación y despliegue. Esto garantiza que la versión publicada esté siempre actualizada con la última versión del código.

5. Propuestas de Mejora

El desarrollo de *MotorsportsLIVE* ha supuesto no solo la implementación de un proyecto completo, sino también un proceso formativo continuo, en el que se ha aprendido a trabajar con herramientas modernas como APIs, Supabase y Node.js. A pesar de haber alcanzado los objetivos iniciales, existen diversas áreas susceptibles de mejora o expansión. A continuación se detallan por ámbitos:

5.1 Funcionalidades Pendientes

Pulido Inicio Sesión	Actualmente, le falla el inicio de sesión, sobre todo al navegar entre páginas, o al acceder como invitado.
Creación de noticias de editores	Posibilidad de crear, editar, o eliminar noticias hechas por los editores registrados.
Añadido de favoritos	Añadir algo como favorito y registrarlo en la base de datos.
Añadir más datos actuales	Poder añadir datos acordes a la temporada actual.
Modo oscuro alternativo	Poder cambiar de color la interfaz de la aplicación. Así como mejorar la interfaz.
Comentarios o votaciones	Añadir una opción de comentarios por eventos, para que cualquier usuario pueda comentar lo que quiera.

5.2 Mejora Técnica del Backend (Supabase y lógica)

Mejora técnica	Justificación
Control de errores más robusto	Capturar fallos en la autenticación, inserción o carga de favoritos para mostrar feedback al usuario. Comprobar si los datos enviados al backend (como URLs) cumplen ciertos criterios antes de ser guardados.
Validación de datos	Aprovechar más filtros o paginación en las llamadas al backend para escalar mejor si aumenta el
Optimización de queries	

volumen de datos.

Auditoría o logs internos

Implementar registro de acciones de usuario (opcional) para análisis posterior del uso.

5.3 Mejora Técnica del Frontend (Estilo visual)

Mejora del código CSS

Unificar estilos comunes en archivos globales y reducir repeticiones.

Mayor modularización de componentes

Algunos componentes pueden dividirse en subcomponentes.

Sistema de notificaciones internas

Feedback al usuario al añadir favoritos, errores de conexión, etc.

6. Webgrafía

6.1. Cursos online y tutoriales utilizados

Plataforma	Curso / Contenido	Autor / Canal
YouTube	Introducción a Supabase y alternativas a Firebase	<i>Midudev</i>
YouTube	<i>Deploy de Backend en Vercel</i>	<i>Pablo Marino</i>
YouTube	Como desplegar y consumir API REST	<i>Roly Dev</i>
YouTube	<i>Despliegue Node.js API REST</i>	<i>Carlos Azaustre</i>

6.2. Webs de referencia

Sitio web	Descripción
Supabase.io	Plataforma backend como servicio (BaaS).
StackOverflow	Comunidad para resolver dudas técnicas.
MDN Web Docs	Referencia completa de tecnologías web.
Node.js	Node.js.
CSS-Tricks	Consejos y técnicas avanzadas de CSS.
Bootstrap	Bootstrap.
Vercel	Vercel.

6.3. Artículos y documentación técnica

- [Supabase Auth Docs](https://supabase.com/docs/guides/auth): guía oficial de autenticación.
- [JWT.io – Introducción a JSON Web Tokens](https://jwt.io): explicación detallada de los tokens de autenticación.

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto:



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo